

dy_engine

프로토타입 구현 결과 발표

Team DyD (Direct, Yield, Dynamic media)

한재승	High Layer Design	20242215
윤 훈	Graphics Dev with Linux	20240901
정현진	Graphics Dev with Win	20242207
정준혁	Graphics Dev with Mac	20240846

발표 내용



01. 개발 현황

개발 완료 내용 소개



02. 체크리스트

무엇을 보면 좋은지
체크리스트 소개



03. 시연

프로토타입 시연



05. 로드맵

추후 개발 내용 소개

04. 패키징

엔진 라이브러리 규모와
패키징 방향

개발 현황

Cross-Platform

모든 기능을 Windows / Linux / Mac에서 검증 완료

Linux의 GUI (X11 / Wayland) 환경 특수성 고려

Model Parser

glTF, FBX, OBJ 포맷 파싱 지원

계층 구조, 메타데이터 유지

CPU 연산 최적화

SoA(Structure of Arrays) 구조로 데이터 통일 후

SIMD 적용된 Math로 병렬 연산

PBR Material

물리 기반 음영 Material 적용

5종의 sub-texture 지원

Post-Processing 확장

렌더링 방식을 여러 시각적 효과의 제공을 위해 확장

Shadow Mapping 구현으로 부드러운 그림자 생성

Render Path

Per-draw, Batched, Bindless 렌더링 모드 분리

사용자 - CPU - GPU 간 고정값 없이 유동적인 렌더 설정

체크리스트 1: SIMD



01. 실제 에셋 입력

glTF, FBX, OBJ 등의 3D 모델 파일
준비 및 엔진 입력 단계



02. 데이터 추출

메쉬 데이터, 머티리얼 속성 및
텍스처 경로 파싱



03. Scene 병합

계층 구조 유지 및 서브메쉬 단위의
씬 데이터 구성



04. SoA 구조 저장

캐시 효율적인 SoA 레이아웃으로
데이터 직렬화 및 저장

SIMD 연산을 위한 align, padding
고려

[Data Structure Overview]

Model file

- Mesh: vertices / indices / normals / tangents / uv
- Material: baseColor / metallic / roughness / emissive
- Texture: baseColor / normal / occlusion / ... → `dy::Graphics::Scene`

체크리스트 2: 렌더링 파이프라인 확장

현재 1

Main Pass 실행

CPU에서 렌더링 데이터를 취합하여 전송하는 메인 쓰레드

현재 2

Draw Call 기록

CommandList가 렌더 명령을 기록 후 GPU로 전송

현재 3

Shader 실행 (GPU)

전송된 렌더 명령을 GPU에서 고속으로 병렬 처리

현재 4

Post-Processing 적용

완성된 2D pixel 데이터를 후처리하고 최종 보정

Post-Processing 확장

효과적인 후처리를 위해 즉시 실행(Draw) 구조 외에 Draw Call 기록 및 후처리 전용 렌더 구조를 추가 제공하였습니다.

기존 방식: 즉시 기록 방식 Draw

- 모델 파일 파싱 / Scene 데이터 구조화

Post-Processing 종류

- **안티앨리어싱 (AA):** 계단 현상 제거 및 경계 보정
- **모션 블러 & 톤 매핑:** 시각적 역동성 및 색감 최적화
- **앰비언트 오클루전:** 주변 폐쇄 효과를 통한 입체감 강화

체크리스트 3: 렌더링 모드 다원화

[테스트 환경]

Hardware

CPU: Ryzen 7 3700X
GPU: GeForce 3500 ti

Scene Info

- Models Loaded: 7
- Template Entities: 10
- Instances: 2,000
- Meshes / Materials: 10 / 9
- Textures: 19

[행렬 연산 성능 (2,000,000 muls 기준)]

SIMD 최적화 적용: 14.68 ms
비적용 (Not SIMD): 31.87 ms

Binding Mode	FPS	해석
Per-draw	30	Draw마다 지오메트리/텍스처 상태 변경 과다
Batched	245	Instanced Draw 적용으로 Draw Call 대폭 감소
Bindless	250	글로벌 힙 인덱싱 대체로 CPU 오버헤드 최소화

프로토타입 시연

주요 확인 포인트



크로스 플랫폼 동작

Windows, Linux, Mac 등 주요 데스크톱 환경에서 동일한 바이너리 인터페이스로 원활하게 동작함을 확인합니다.



바인딩 모드별 **FPS**

Per-draw 방식에서 Bindless 렌더링으로 전환 시 발생하는 프레임 속도 (FPS)의 실시간 변화를 시뮬레이션합니다.



SIMD 연산 수치

대량의 행렬 연산 처리 시 SIMD 최적화 여부에 따른 연산 효율 및 소요 시간의 유의미한 수치를 비교 분석합니다.

발견된 오류

D3D12 Backend Implementation

```
현재 D3D12Device::Present()  
mandQueue->Signal(fence, value);  
(fence->GetCompletedValue() < value) {  
    fence->SetEventOnCompletion(value, fenceEvent  
    WaitForSingleObject(fenceEvent, INFINITE);
```

Vulkan Context Structure

```
// src/Backends/Vulkan/VulkanContext.h  
#define VK_USE_PLATFORM_WIN32_KHR  
#include <vulkan/vulkan.h>  
  
struct VulkanContext {  
    VkInstance instance = VK_NULL_HANDLE;  
    VkDevice device = VK_NULL_HANDLE;  
    VkQueue graphicsQueue = VK_NULL_HANDLE;  
};
```



발견된 기술 부채

- **D3D12 동기화 병목:** Present 단계에서 GPU 완료를 매 프레임 대기하여 병렬성이 제한됨 (frames-in-flight 개선 필요)
- **Vulkan 헤더 노출:** vulkan.h 가 헤더 경계로 확산되어 RHI의 API 종립성을 저해함 (타입 은닉 필요)

엔진 규모 및 패키징 아키텍처

항목	DirectX12	Vulkan MSVC(windows)	Vulkan GCC(linux)	Metal
dy_engine [.lib .a]	5.63 MB	6.39 MB	5.91 MB	3.10 MB
ufbx 외부 FBX 로더	1.19 MB	1.13 MB	1.08 MB	1.01 MB
별도 fastgltf 라이브러리	6.50 MB	7.86 MB	6.19 MB	2.90 MB



패키징 전략

Model Loader와 Socket 통신 기능을 코어에서 분리하여
렌더링 코어는 가볍게 유지하고 필요한 기능만 선택적으로 링크하도록 설계합니다.

Loadmap



Render Graph

렌더 패스와 리소스 의존성을 그래프로 선언하여 Shadow, Post-process 등이 늘어날 때 배리어와 상태 전이를 자동화합니다.



Multi-Thread

스레드별 커맨드 기록과 frames-in-flight 구조를 개선하여 현대 API가 의도한 CPU 병렬성을 활용하고 제출 병목을 완화합니다.



Compute & Mesh

그래픽스 파이프라인 밖의 범용 GPU 연산(Compute)을 RHI에 통합하고 차세대 지오메트리 처리(Mesh Shader)로 확장합니다.

Q&A

지금까지 dy_engine 프로토타입 발표였습니다.